GlobalPay

International Money Transfer Platform

System Design Document

HLD | LLD | Spring Boot 3

Framework:	Spring Boot 3.3 + Java 17
Databases:	PostgreSQL 16 · Redis · Cassandra
Messaging:	Apache Kafka
Version:	1.0 | February 2026

1. Problem Statement

GlobalPay is an international peer-to-peer (P2P) money transfer platform inspired by Western Union that enables users across multiple countries to send funds using credit/debit cards. The system must be intelligent, personalised, and high-performance.

- Users in different countries perform cross-border transactions with varying currencies, exchange rates, and regulatory requirements.
- Identify and surface **priority transactions** (high-value, time-sensitive, VIP users) and route them through faster processing lanes.
- Coupon and discount engine must reward loyal and frequent users based on transaction history, destination, and user segment.
- Credit card processing must handle international card networks (Visa, Mastercard, Amex) with 3DS, fraud detection, and chargeback management.
- Latency target: **< 2s UI response**, **< 30s settlement initiation**.
- Must be horizontally scalable, fault-tolerant, AML and KYC compliant.

2. High-Level Design (HLD)

2.1 Architecture Overview

GlobalPay adopts a **Microservices Architecture** deployed on Kubernetes with an API Gateway as single entry point. Each domain is an independent service with its own database, communicating via REST (sync) and Apache Kafka (async event streaming).

Layer	Component	Technology	Responsibility
Client	Mobile / Web App	React Native / Next.js	User-facing frontend
Gateway	API Gateway	Spring Cloud Gateway	Auth, rate limiting, routing
Auth	Identity Service	Spring Security + OAuth2	Login, MFA, KYC status
Core	Transaction Service	Spring Boot 3	Initiate, validate, route TXN
Core	Payment Service	Spring Boot 3 + Stripe	Credit card charging
Core	FX Service	Spring Boot 3 + Redis	Real-time exchange rates
Core	Priority Engine	Spring Boot 3 + Redis	VIP/high-value TXN scoring
Core	Coupon Service	Spring Boot 3	Coupon generation & redemption
Core	Notification Service	Spring Boot + Firebase	Push/SMS/Email alerts
Core	Compliance Service	Spring Boot 3	AML, sanctions screening
Data	User + TXN DB	PostgreSQL 16	Financial records, KYC, ledger
Data	Cache	Redis Cluster	Sessions, FX rates, priority scores

Layer	Component	Technology	Responsibility
Data	Event Stream	Apache Kafka	Async event processing
Data	Analytics DB	Apache Cassandra	TXN analytics, audit logs
Infra	Service Mesh	Istio	mTLS, circuit breaking, tracing

2.2 Priority Transaction System

Priority scoring determines which transactions use the **fast lane** (target <5s settlement) vs standard queue.

Priority	Score	Criteria	Processing
PLATINUM	90–100	VIP user OR amount >$10,000 OR business account	Dedicated thread pool, skip queue
GOLD	70–89	Premium user OR amount $1,000–$10,000	High-priority Kafka partition
SILVER	50–69	Regular user ≥3 TXN/month OR $200–$999	Standard queue, prioritised
STANDARD	0–49	New or infrequent user, low amount	Standard processing queue

2.3 Coupon & Reward Engine

- **Rule-based triggers:** First international transfer, milestone TXNs (5th, 10th, 25th), birthday month.
- **Corridor offers:** Users on high-frequency corridors (US→India, UK→Philippines) get corridor-specific coupons.
- **Dynamic fee discounts:** 0% fee when 30-day volume exceeds threshold; tier-based percentage discounts.
- **Referral coupons:** Sender gets 50% fee discount when referred user completes first transaction.
- **Expiry management:** 7–30 day TTL; 48h reminder notification before expiry.

3. Low-Level Design (LLD)

3.1 Transaction Service - Saga Orchestrator

The TransactionService is the central orchestrator. It coordinates all downstream services using the **Saga pattern** with compensating transactions on failure.

Java

```java
// TransactionService.java  -  Saga Orchestrator (Part 1: Setup & Steps 1-4)
@Service
@RequiredArgsConstructor
@Slf4j
public class TransactionService {

    private final FxService            fxService;
    private final PriorityEngine       priorityEngine;
    private final CouponService        couponService;
    private final ComplianceService    complianceService;
    private final PaymentGateway       paymentGateway;
    private final TransactionRepository txnRepo;
    private final OutboxRepository     outboxRepo;
    private final ObjectMapper         objectMapper;

    @Transactional
    public TransactionResponse initiateTransfer(
            TransactionRequest req, String userId) {

        // -- Step 1: Fetch FX rate (Redis-cached, TTL 30s) ---------------
        FxRate rate = fxService.getRate(
                req.getSourceCurrency(), req.getTargetCurrency());

        // -- Step 2: Score priority --------------------------------------
        int priorityScore = priorityEngine.score(userId, req.getAmount());
        PriorityLevel priority = PriorityLevel.fromScore(priorityScore);
        log.info("TXN priority=[{}] score=[{}] user=[{}]",
                 priority, priorityScore, userId);

        // -- Step 3: Apply best applicable coupon ------------------------
        CouponResult coupon = couponService.apply(userId, req);
        BigDecimal fee = coupon.apply(calculateFee(req.getAmount()));

        // -- Step 4: Compliance check ------------------------------------
        if (priority.requiresSyncCompliance()) {
            complianceService.screenSync(req, userId);   // PLATINUM / GOLD
        }
```

3.2 Priority Engine

Java

```java
// PriorityEngine.java  -  Scores each transaction 0–100
@Service
@RequiredArgsConstructor
public class PriorityEngine {

    private static final String CACHE_PREFIX = "txn:count:";

    private final RedisTemplate<String, Integer> redis;
    private final UserRepository                 userRepo;
    private final TransactionRepository          txnRepo;

    public int score(String userId, BigDecimal amount) {
        User user  = userRepo.findById(userId).orElseThrow();
        int  score = 0;

        // -- Amount scoring (max 40 pts) ----------------------------------
        if (amount.compareTo(BigDecimal.valueOf(10_000)) >= 0) score += 40;
        else if (amount.compareTo(BigDecimal.valueOf(1_000)) >= 0) score += 25;
        else if (amount.compareTo(BigDecimal.valueOf(200))   >= 0) score += 10;

        // -- User-tier scoring (max 30 pts) -------------------------------
        score += switch (user.getTier()) {
            case VIP      -> 30;
            case PREMIUM  -> 20;
            case STANDARD -> 10;
            default       -> 0;
        };

        // -- Monthly TXN frequency (max 20 pts) - Redis-cached 1h ---------
        Integer count = redis.opsForValue().get(CACHE_PREFIX + userId);
        if (count == null) {
            count = txnRepo.countByUserIdSince(userId,
                    LocalDate.now().minusMonths(1));
            redis.opsForValue().set(CACHE_PREFIX + userId, count,
                    Duration.ofHours(1));
        }
        if (count >= 10) score += 20;
        else if (count >= 3) score += 10;

        // -- Business account bonus (max 10 pts) --------------------------
        if (user.getAccountType() == AccountType.BUSINESS) score += 10;

        return Math.min(score, 100);
    }
}
```

3.3 FX Service with Redis Caching & Circuit Breaker

Java

```java
// FxService.java  -  Exchange rate provider with resilience
@Service
@RequiredArgsConstructor
public class FxService {

    private final RedisTemplate<String, FxRate> redis;
    private final ExternalFxClient              fxClient;

    @CircuitBreaker(name = "fxService", fallbackMethod = "cachedRate")
    @Retry(name = "fxService")
    public FxRate getRate(String from, String to) {
        String key   = "fx:" + from + ":" + to;
        FxRate cached = redis.opsForValue().get(key);
        if (cached != null) return cached;

        FxRate rate = fxClient.fetchRate(from, to);
        redis.opsForValue().set(key, rate, Duration.ofSeconds(30));
        return rate;
    }

    // Fallback: return last known cached rate on external API failure
    public FxRate cachedRate(String from, String to, Exception ex) {
        FxRate stale = redis.opsForValue().get("fx:" + from + ":" + to);
        if (stale != null) return stale;
        throw new FxUnavailableException("FX rate unavailable for " + from + "/" + to);
    }

    // Warm-up job: refresh top corridors every 30 seconds
    @Scheduled(fixedDelay = 30_000)
    public void refreshTopCorridors() {
        List.of("USD:INR", "USD:PHP", "GBP:EUR", "USD:MXN",
                "USD:NGN", "GBP:USD", "EUR:GBP", "USD:CAD")
            .forEach(corridor -> {
                String[] parts = corridor.split(":");
                FxRate   rate  = fxClient.fetchRate(parts[0], parts[1]);
                redis.opsForValue().set("fx:" + corridor, rate,
                                        Duration.ofSeconds(60));
            });
    }
}
```

3.4 Coupon Service - Strategy Pattern

Java

```java
// CouponStrategy.java - Contract for all coupon evaluators
public interface CouponStrategy {
    Coupon evaluate(String userId, TransactionRequest req);
}

// CouponService.java - Picks the best applicable coupon
@Service
@RequiredArgsConstructor
public class CouponService {

    private final List<CouponStrategy> strategies;
    private final CouponRepository     couponRepo;

    public CouponResult apply(String userId, TransactionRequest req) {
        List<Coupon> applicable = strategies.stream()
            .map(strategy -> strategy.evaluate(userId, req))
            .filter(Objects::nonNull)
            .filter(c -> !couponRepo.isUsedByUser(c.getCode(), userId))
            .sorted(Comparator.comparing(Coupon::getDiscountRate).reversed())
            .toList();

        if (applicable.isEmpty()) return CouponResult.none();

        Coupon best = applicable.get(0);
        couponRepo.markUsed(best.getCode(), userId);
        return CouponResult.of(best);
    }
}
```

Java

```java
// -- Strategy Implementations (auto-discovered by Spring) --------------------

@Component
public class FirstTransferCouponStrategy implements CouponStrategy {
    public Coupon evaluate(String userId, TransactionRequest req) {
        long count = txnRepo.countByUserId(userId);
        return count == 0 ? Coupon.zeroFee("FIRST-TXN-FREE") : null;
    }
}

@Component
public class CorridorCouponStrategy implements CouponStrategy {
    private static final Set<String> TOP_CORRIDORS =
        Set.of("USD:INR", "USD:PHP", "GBP:NGN", "EUR:USD");

    public Coupon evaluate(String userId, TransactionRequest req) {
        String corridor = req.getSourceCurrency() + ":" + req.getTargetCurrency();
        return TOP_CORRIDORS.contains(corridor)
            ? Coupon.percentDiscount("CORRIDOR-50", 50) : null;
    }
}

@Component
public class LoyaltyCouponStrategy implements CouponStrategy {
    public Coupon evaluate(String userId, TransactionRequest req) {
        long count = txnRepo.countByUserId(userId);
        if (count == 9)  return Coupon.percentDiscount("LOYALTY-10TH", 25);
        if (count == 24) return Coupon.percentDiscount("LOYALTY-25TH", 50);
        if (count == 49) return Coupon.zeroFee("LOYALTY-50TH");
        return null;
    }
}
```

4. Database Design

4.1 Polyglot Persistence Strategy

Database	Type	Use Case	Why?
PostgreSQL 16	Relational (ACID)	Users, Transactions, Cards, Coupons, Ledger	Strong consistency; ACID; partitioning; native JSON
Redis Cluster	In-Memory K/V	FX cache, sessions, priority scores, rate limiting, idempotency	Sub-ms reads; TTL; atomic Lua scripts; Pub/Sub
Apache Cassandra	Wide-Column NoSQL	Analytics, audit logs, notification history, activity timeline	Linear horizontal scale; optimised for time-series writes

4.2 PostgreSQL Schema

SQL

```sql
-- -- Users ---------------------------------------------------------------------
CREATE TABLE users (
    id            UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    email         VARCHAR(255) UNIQUE NOT NULL,
    full_name     VARCHAR(255) NOT NULL,
    country_code  CHAR(2)     NOT NULL,
    account_type  VARCHAR(20) NOT NULL DEFAULT 'STANDARD',
    tier          VARCHAR(20) NOT NULL DEFAULT 'STANDARD',
    kyc_status    VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- -- Transactions (range-partitioned by month) ---------------------------------
CREATE TABLE transactions (
    id              UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id         UUID          NOT NULL REFERENCES users(id),
    recipient_id    UUID          REFERENCES users(id),
    amount          NUMERIC(18,2) NOT NULL,
    source_currency CHAR(3)       NOT NULL,
    target_currency CHAR(3)       NOT NULL,
    fx_rate         NUMERIC(12,6) NOT NULL,
    fee             NUMERIC(10,2) NOT NULL DEFAULT 0,
    coupon_code     VARCHAR(50),
    priority_level  VARCHAR(20)   NOT NULL DEFAULT 'STANDARD',
    status          VARCHAR(30)   NOT NULL DEFAULT 'PENDING',
    payment_ref     VARCHAR(255),
    created_at      TIMESTAMPTZ   NOT NULL DEFAULT NOW()
) PARTITION BY RANGE (created_at);

CREATE TABLE transactions_2026_02 PARTITION OF transactions
    FOR VALUES FROM ('2026-02-01') TO ('2026-03-01');

CREATE INDEX idx_txn_user_created ON transactions(user_id, created_at DESC);
CREATE INDEX idx_txn_status       ON transactions(status)
    WHERE status IN ('PENDING', 'PROCESSING');
```

```sql
-- -- Coupons ---------------------------------------------------------------
CREATE TABLE coupons (
    code          VARCHAR(50)   PRIMARY KEY,
    type          VARCHAR(30)   NOT NULL,  -- ZERO_FEE | PERCENT_DISCOUNT | FLAT
    discount_rate NUMERIC(5,2),
    valid_from    DATE          NOT NULL,
    valid_until   DATE          NOT NULL,
    max_uses      INTEGER,
    used_count    INTEGER       NOT NULL DEFAULT 0,
    version       INTEGER       NOT NULL DEFAULT 0,  -- optimistic locking
    corridor      VARCHAR(10),   -- 'USD:INR' or NULL = all corridors
    user_segment  VARCHAR(30)    -- NULL = all users
);

-- -- User coupon usage (prevents double-redemption) ----------------------------
CREATE TABLE user_coupon_usage (
    user_id        UUID        NOT NULL REFERENCES users(id),
    coupon_code    VARCHAR(50) NOT NULL REFERENCES coupons(code),
    used_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (user_id, coupon_code)
);

-- -- Outbox (guaranteed Kafka delivery) -----------------------------------
CREATE TABLE outbox_events (
    id         UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
    topic      VARCHAR(100) NOT NULL,
    payload    TEXT         NOT NULL,
    processed  BOOLEAN      NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_outbox_pending ON outbox_events(created_at)
    WHERE processed = FALSE;
```

4.3 Database Design Patterns

Pattern	Applied Where	Benefit
Table Partitioning	transactions (by month)	Query perf, easy archival, partition pruning
Repository Pattern	All Spring Data JPA repos	Decouples business logic from persistence
Outbox Pattern	TXN events → Kafka via DB outbox	Guaranteed event delivery, no dual-write
CQRS	Write to PostgreSQL, read analytics from Cassandra	Independent read/write scaling
Optimistic Locking	coupon.version via @Version	Prevent race conditions on coupon redemption
Read Replicas	PostgreSQL primary + 2 replicas	Route read queries away from primary
Idempotency Key	Redis SETNX with 24h TTL	Prevent duplicate charges on retries
Unit of Work	Spring @Transactional	Atomic multi-table ops, rollback on failure

5. Design Patterns Summary

Pattern	Category	Where Used	Why
Saga (Orchestration)	Distributed	TransactionService	No 2PC; compensating actions
Strategy	Behavioural	CouponService strategies	Open/Closed; pluggable coupons
Factory	Creational	PaymentGateway implementations	Decouple from concrete processors
Builder	Creational	Transaction, FxRate, CouponResult	Readable complex object creation
Observer / Event	Behavioural	Kafka events post-TXN	Decoupled notifications & analytics
Circuit Breaker	Resilience	FX API + card network calls	Fail fast; fallback to cache
Decorator	Structural	Fee calculation pipeline	Stack transformations cleanly
Repository	Data Access	All JPA repositories	Testable persistence abstraction
Outbox	Data	Reliable Kafka publishing	Event delivery matches DB state
CQRS	Architectural	Read/write separation	Independent scaling of paths
Proxy / AOP	Structural	@Transactional, @Cacheable, logging	Cross-cutting concerns

5.1 Outbox Pattern - Guaranteed Kafka Delivery

Java

```java
// OutboxPoller.java  -  Transactional relay from DB to Kafka
@Component
@RequiredArgsConstructor
@Slf4j
public class OutboxPoller {

    private final OutboxRepository                    outboxRepo;
    private final KafkaTemplate<String, String>       kafka;

    @Scheduled(fixedDelay = 1_000)   // poll every 1 second
    @Transactional
    public void relay() {
        List<OutboxEvent> pending =
            outboxRepo.findTop50ByProcessedFalseOrderByCreatedAtAsc();

        pending.forEach(event -> {
            try {
                kafka.send(event.getTopic(), event.getId().toString(),
                           event.getPayload()).get();       // sync send
                event.setProcessed(true);
                log.debug("Relayed outbox event [{}] to topic [{}]",
                          event.getId(), event.getTopic());
            } catch (Exception ex) {
                log.error("Failed to relay outbox event [{}]: {}",
                          event.getId(), ex.getMessage());
                // will retry on next poll
            }
        });

        outboxRepo.saveAll(pending);
    }
}
```

6. Latency Reduction Strategies

Strategy	Saving	Implementation
Redis FX Rate Cache (TTL 30s)	~200ms	Avoid external API per request; scheduled warm-up
Priority-based thread pools	Variable	Spring @Async + dedicated pools per tier
Idempotency key pre-check	~500ms	Redis SETNX before any DB write
Async compliance (STANDARD)	~800ms off critical path	Kafka consumer post-charge for standard tier
HikariCP connection pooling	~50ms	Pre-warmed connections; pool tuned to DB cores
Read replicas	~30ms	@Transactional(readOnly=true) routed to replicas
Kafka priority partitions	~100ms queue wait	PLATINUM=partition 0, GOLD=1–2, STD=3–9
Keyset pagination	~150ms	No COUNT(*) on large tables; cursor-based listing

6.1 HikariCP + Async Thread Pool Config

Java

```
# application.yml - HikariCP & Resilience4j config

spring:
  datasource:
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 3000       # 3s max wait for connection
      idle-timeout: 600000           # 10 min
      max-lifetime: 1800000          # 30 min
      keepalive-time: 60000          # ping DB every 60s

resilience4j:
  circuitbreaker:
    instances:
      fxService:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
      paymentGateway:
        slidingWindowSize: 20
        failureRateThreshold: 30
        waitDurationInOpenState: 30s
```

Java

```java
// AsyncConfig.java - Priority-based thread pools
@Configuration @EnableAsync
public class AsyncConfig {

    @Bean("platinumExecutor")
    public Executor platinumExecutor() {
        return buildPool("platinum-", 10, 20, 50);
    }

    @Bean("goldExecutor")
    public Executor goldExecutor() {
        return buildPool("gold-", 15, 40, 200);
    }

    @Bean("standardExecutor")
    public Executor standardExecutor() {
        return buildPool("standard-", 30, 80, 500);
    }

    private Executor buildPool(String prefix, int core, int max, int queue) {
        ThreadPoolTaskExecutor e = new ThreadPoolTaskExecutor();
        e.setCorePoolSize(core);
        e.setMaxPoolSize(max);
        e.setQueueCapacity(queue);
        e.setThreadNamePrefix(prefix);
        e.setWaitForTasksToCompleteOnShutdown(true);
        e.initialize();
        return e;
    }
}
```

7. Non-Functional Requirements & Security

Requirement	Target	Strategy
API Response (P95)	< 2 seconds	Redis caching, async compliance, SQL tuning
Settlement (PLATINUM)	< 5 seconds	Dedicated threads, sync compliance, Kafka priority
Availability	99.95%	Multi-AZ K8s, health checks, circuit breakers
Throughput	>5,000 TPS peak	HPA, Kafka partitioning, DB read replicas
Data Durability	99.9999%	PostgreSQL sync replication, PITR backups
PCI-DSS	Level 1	Stripe tokenisation; no card data in GlobalPay DB
GDPR	Full compliance	AES-256 PII encryption; data deletion API; audit log

- **JWT:** 15-minute access tokens; refresh tokens in Redis with sliding expiry.
- **mTLS:** All service-to-service traffic encrypted via Istio; zero-trust internal network.
- **3DS 2.0:** Enforced for all international transactions > $100.
- **Rate limiting:** 100 req/min per user; 10 TXN attempts/hour at API Gateway.
- **PII encryption:** AES-256 column-level encryption with AWS KMS envelope keys.
- **Audit log:** Append-only Cassandra table; every state change immutably recorded.

8. Key Architectural Decisions

Decision	Choice	Rationale
Architecture	Microservices on Kubernetes	Independent scaling; resilience; team autonomy
Language	Java 17 + Spring Boot 3.3	Virtual threads; records; mature fintech ecosystem
Primary DB	PostgreSQL 16 + partitioning	ACID; excellent JSON; proven in fintech at scale
Cache	Redis Cluster	Sub-ms reads; TTL; distributed locking; pub/sub
Analytics DB	Apache Cassandra	Linear scale; time-series optimised; 99.99% HA
Messaging	Apache Kafka	Durable log; priority partitioning; exactly-once
Distributed TXN	Saga Orchestration	No 2PC; each step has compensating action
Payment	Stripe + Visa Direct	PCI-DSS handled externally; tokenisation
Resilience	Resilience4j	Native Spring Boot 3 support; fallback to cache
Security	OAuth2 + JWT + mTLS	Stateless auth; zero-trust via Istio

GlobalPay is designed to be **user-centric** (priority lanes), **rewarding** (intelligent coupons), **latency-optimised** (<2s P95), **financially safe** (Saga + Outbox + idempotency), and **compliance-ready** (AML + KYC from day one).